

1_01_INTRODUCTION_Overall_Architecture

Introduction to the Overall Architecture

Human Summary

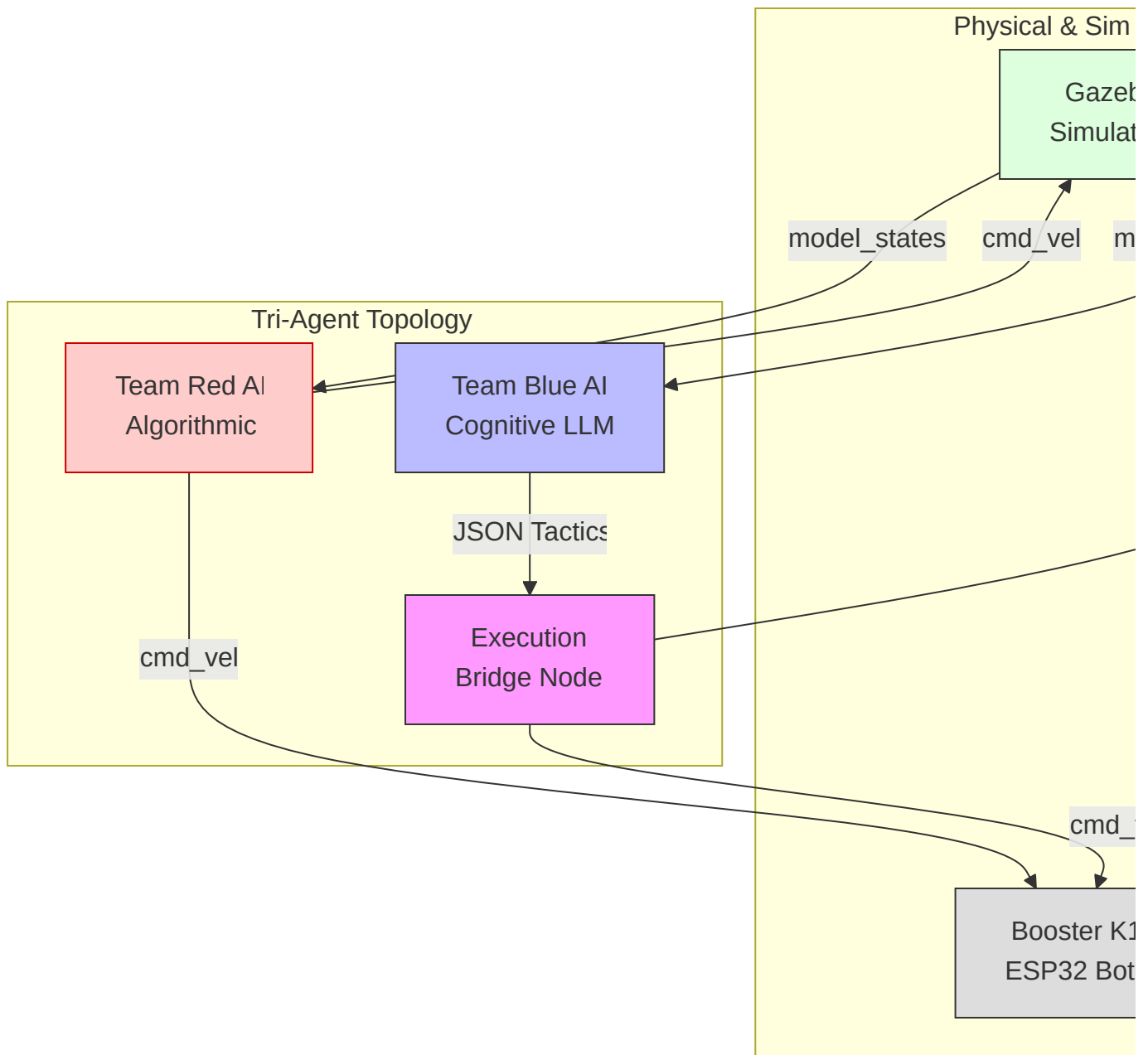
This document defines the foundational scope of the ROS2K project, outlining its dual-purpose as a RoboCup simulation environment and a low-latency LLM robotics workbench. It establishes the baseline tri-agent topology (Team Blue, Team Red, Hardware) required to understand the system.

LLM Context Anchor

ROS2K explicitly rejects rigid C++ Hardware Abstraction Layers (HALs). It enforces strict decoupling between high-frequency physical execution (ROS 2/Gazebo) and low-frequency cognitive strategy (Nemotron-3-nano:4b) via asynchronous File I/O polling.

1. System Topology of the Hybrid Environment

The following graph illustrates the macroscopic separation of concerns between physical simulation, algorithmic baselines, and the asynchronous cognitive LLM layer.



2. Architectural Logic & Data Flow

The ROS2K architecture resolves conflicting operational paradigms by serving two concurrent objectives:

1. **RoboCup Development Environment:** Integrates Gazebo, ROS 2 topics, and bidirectional UDP multicast (FastDDS) to orchestrate mixed-reality scenarios. It provides a standardized `/cmd_vel` ingestion pipeline for both simulated differential-drive bots and physical bipeds.
2. **LLM Robotics Workbench:** Provides a constrained testbed optimizing small-parameter LLMs (Nemotron-3-nano:4b). By isolating the LLM from ROS 2 middleware via a JSON file-system multiplexer, the architecture prevents the Global Interpreter Lock (GIL) and ROS 2 spin loops from blocking during 500-2000ms inference pulses.

Team Red serves as the deterministic control group, utilizing hardcoded Python state machines to evaluate spatial distance instantaneously. Team Blue executes natural language

spatial reasoning, sacrificing latency for complex strategy generation.

3. Code Reference & Interfaces

Source: [launch_r2k.sh](#)

The core environment bootstrap logic forces FastDDS network binding to the host OS to ensure physical edge hardware (ESP32) can receive UDP multicast packets from the Docker container.

```
# snippet from launch_r2k.sh
export ROS_DOMAIN_ID=42
export FASTRTPS_DEFAULT_PROFILES_FILE=/root/ros2k_unify2/fastdds.xml

# Launch Gazebo simulation alongside detached Python background daemons
ros2 launch r2k_scenario_spawner empty_soccer_pitch.launch.py &
python3 ai_tactics/r2k_evaluator.py &
ros2 run ai_tactics ollama_sandbox_bridge
```

4. Known Issues & Limitations

- Sim2Real localization gap requires dead-reckoning fallbacks due to missing absolute `/gazebo/model_states` in the physical world.
- LLM token generation limits responsiveness to rapid physical ball deflections.

5. Glossary

- **Tri-Agent Topology:** The categorization of entities into Simulated/Real Hardware, Algorithmic AI (Red), and Cognitive AI (Blue).
- **Sim2Real:** Simulation-to-Reality transfer.
- **FastDDS:** The default Data Distribution Service for ROS 2 Humble, reliant on UDP multicast.